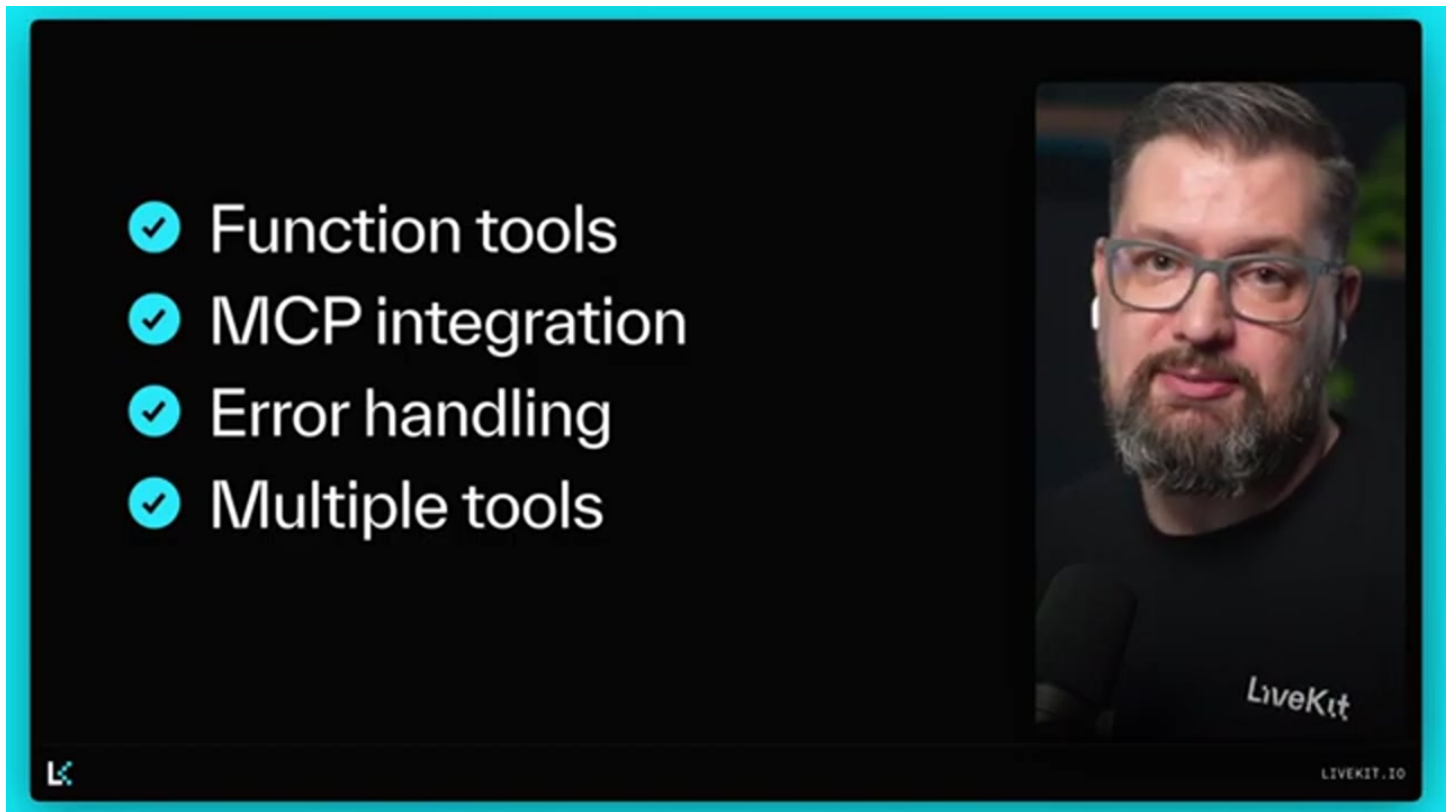


Lesson 6: Consent and Escalations

Section: LiveKit Voice AI Course

Summary: Coverage of Lesson 6: Consent and Escalations (56:36 - 01:09:19)

■ [56:36 - 56:56]



human agents. Your agent has tools. It handles failures gracefully, and you can measure its performance. For production deployments, you'll also need to handle operational workflows like collecting consent and escalating to human agents when needed. Realworld voice agents need to do more than just answer questions.

Production workflows



LIVEKIT.IO

to do more than just answer questions. You need to collect recording consent before capturing calls, which is often legally required. You need to escalate to humans when the AI can't help or uh the user requests it. You need to handoff between specialized agents based on conversational context. And you need to preserve context across these

Production workflows

- Collect recording consent
- Escalate to humans
- Handoff between agents
- Preserve context



LIVEKIT.IO

to preserve context across these transitions. So let's build these capabilities step by step. Tasks are focused units of work that complete and return a result. They're perfect for structured interactions like consent collection where you need a clear yes or no answer before proceeding. So let's build a task that collects recording

Understanding tasks



LIVEKIT.IO

build a task that collects recording consent. We're going to add another uh import here from live kit agents uh live agent task. So next we're going to define the consent task. So we have a new class here, collect consent, which extends the agent task. And so this is going to return a boolean true or false.

■ [57:56 - 58:16]

```
livekit-voice-agent > agent.py

10 from livekit.agents import function_tool, RunContext, ToolError
11 from livekit.agents import mcp
12 from livekit.agents import AgentTask
13 import time
14 import httpx
15
16 logger = logging.getLogger(__name__)
17
18 load_dotenv()
19
20 class CollectConsent(AgentTask[bool]):
21     def __init__(self, chat_ctx=None):
22         super().__init__(
23             instructions="""
24             Ask for recording consent and get a clear yes or no answer.
25             Be polite and professional.
26             """,
27             chat_ctx=chat_ctx,
28         )
29
30     async def on_enter(self) -> None:
31         await self.session.generate_reply(
32             instructions="""
33             Briefly introduce yourself, then ask for permission to record
```



The instructions ask for recording consent and get a clear yes or no answer. Be polite and professional. That's important. On enter, this method starts when the task starts. And so there's instructions here. Briefly introduce yourself and then ask for permission to record the call for quality assurance and training purposes. Make it clear that they can decline. And

■ [58:16 - 58:36]

```
20 class CollectConsent(AgentTask[bool]):
21     def __init__(self, chat_ctx=None):
22         super().__init__(
23             chat_ctx=chat_ctx,
24         )
25
26     async def on_enter(self) -> None:
27         await self.session.generate_reply(
28             instructions="""
29             Briefly introduce yourself, then ask for permission to record
30             the call for quality assurance and training purposes.
31             Make it clear that they can decline.
32             """
33         )
34
35     @function_tool()
36     async def consent_given(self) -> None:
37         """Use this when the user gives consent to record."""
38         self.complete(True)
39
40     @function_tool()
41     async def consent_denied(self) -> None:
42         """Use this when the user denies consent to record."""
43         self.complete(False)
```



Make it clear that they can decline. And then we have two function tools. We have consent given and consent declined. And it basically just returns true or false. So now we can use this task in our main agent. So now if we go into our assistant first, let's go ahead and update our instructions. We're going to change these from our last agent. Uh you are a friendly customer service representative. Very simple. And then we

■ [58:36 - 58:56]

```
py ●
ice-agent > agent.py

class CollectConsent(AgentTask[bool]):
    async def consent_denied(self) -> None:
        self.complete(False)

class Assistant(Agent):
    def __init__(self) -> None:
        super().__init__(
            instructions="You are a friendly customer service representative."
        )

    @function_tool()
    async def lookup_weather(
        self,
        context: RunContext,
        location: str,
    ) -> dict:
        """Look up current weather for a location.

        Args:
            location: City name or location to get weather for.
        """
```



representative. Very simple. And then we don't need any of these other tools uh that we used in our last lesson. So we'll just go ahead and replace all of this. So here is our new uh assistant class. So you're a friendly customer service representative on enter collect consent. If we have consent then thank

■ [58:56 - 59:16]

it-voice-agent > agent.py

```
class CollectConsent(AgentTask[bool]):
    async def consent_denied(self) -> None:

class Assistant(Agent):
    def __init__(self) -> None:
        super().__init__(
            instructions="You are a friendly customer service representative."
        )

    async def on_enter(self) -> None:
        consent = await CollectConsent(chat_ctx=self.chat_ctx)

        if consent:
            await self.session.generate_reply(
                instructions="Thank them and offer your assistance."
            )
        else:
            await self.session.generate_reply(
                instructions="Let them know you understand and will proceed without recording."
            )

server = AgentServer()
```



consent. If we have consent then thank them and offer them your assistance. Uh if they declined consent let them know that you understand and you will proceed without recording. So this pattern keeps consent collection clean and separate from your main agent logic. So let's go ahead and try this out. Make sure you save it. Open up your terminal and let us run the agent in the console.

■ [59:16 - 59:36]

```
livekit-voice-agent > agent.py

20 class CollectConsent(AgentTask[bool]):
47     self.complete(False)
48
49 class Assistant(Agent):
50     def __init__(self) -> None:
51         super().init (

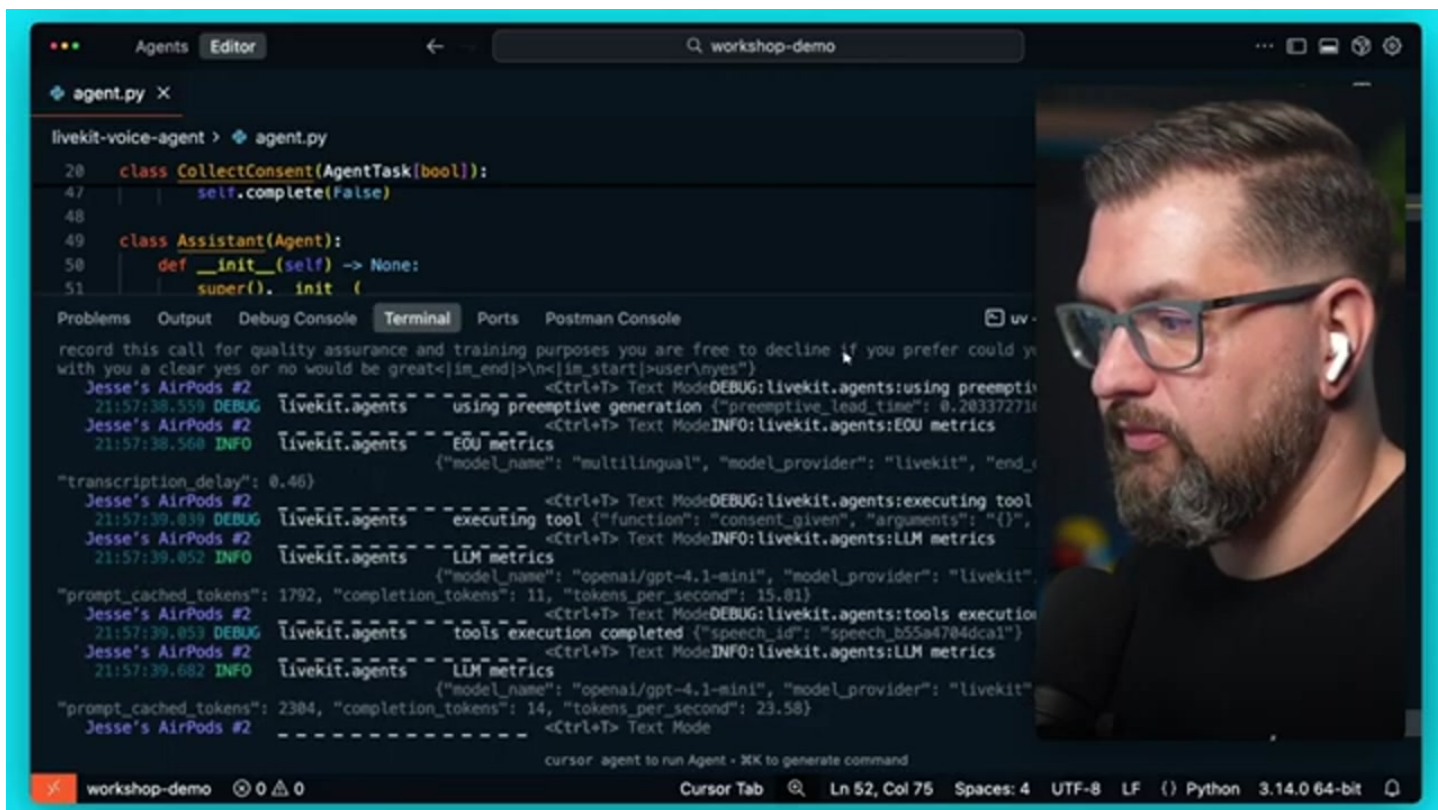
Problems  Output  Debug Console  Terminal  Ports  Postman Console

21:57:16.408 INFO livekit.agents starting inference executor
INFO:livekit.agents:initializing process
21:57:16.416 INFO livekit.agents initializing process {"pid": 64367, "inference": t
INFO:livekit.agents:process initialized
21:57:17.999 INFO livekit.agents process initialized {"pid": 64367, "inference": tr
INFO:livekit.agents:HTTP server listening on :54873
21:57:18.000 INFO livekit.agents HTTP server listening on :54873
INFO:livekit.agents:initializing job runner
DEBUG:asyncio:Using selector: KqueueSelector
21:57:18.011 INFO livekit.agents initializing job runner {"tid": 38786356}
DEBUG:asyncio Using selector: KqueueSelector
INFO:livekit.agents:job runner initialized
21:57:18.012 INFO livekit.agents job runner initialized {"tid": 38786356, "elapsed_
DEBUG:livekit.agents:http_session(): creating a new httpclient ctx
21:57:18.130 DEBUG livekit.agents http_session(): creating a new httpclient ctx
Jesse's AirPods #2 ----- <Ctrl+T> Text ModeDEBUG:livekit.agents:using audio io: `Console`
e`
21:57:18.748 DEBUG livekit.agents using audio io: `Console` -> `AgentSession` -> `Console`
Jesse's AirPods #2 ----- <Ctrl+T> Text ModeDEBUG:livekit.agents:using transcript io: `Ager
DEBUG livekit.agents using transcript io: `AgentSession` -> (none)
Jesse's AirPods #2 ----- <Ctrl+T> Text Mode
```



us run the agent in the console. >> Hello. Before we proceed, I'd like to introduce myself. I'm your assistant here to help with any questions or support you need. May I have your permission to record this call for quality assurance and training purposes? You are free to decline if you prefer. Could you please let me know if that is okay with you? A clear yes or no would be great. >> Yes.

■ [59:36 - 59:56]



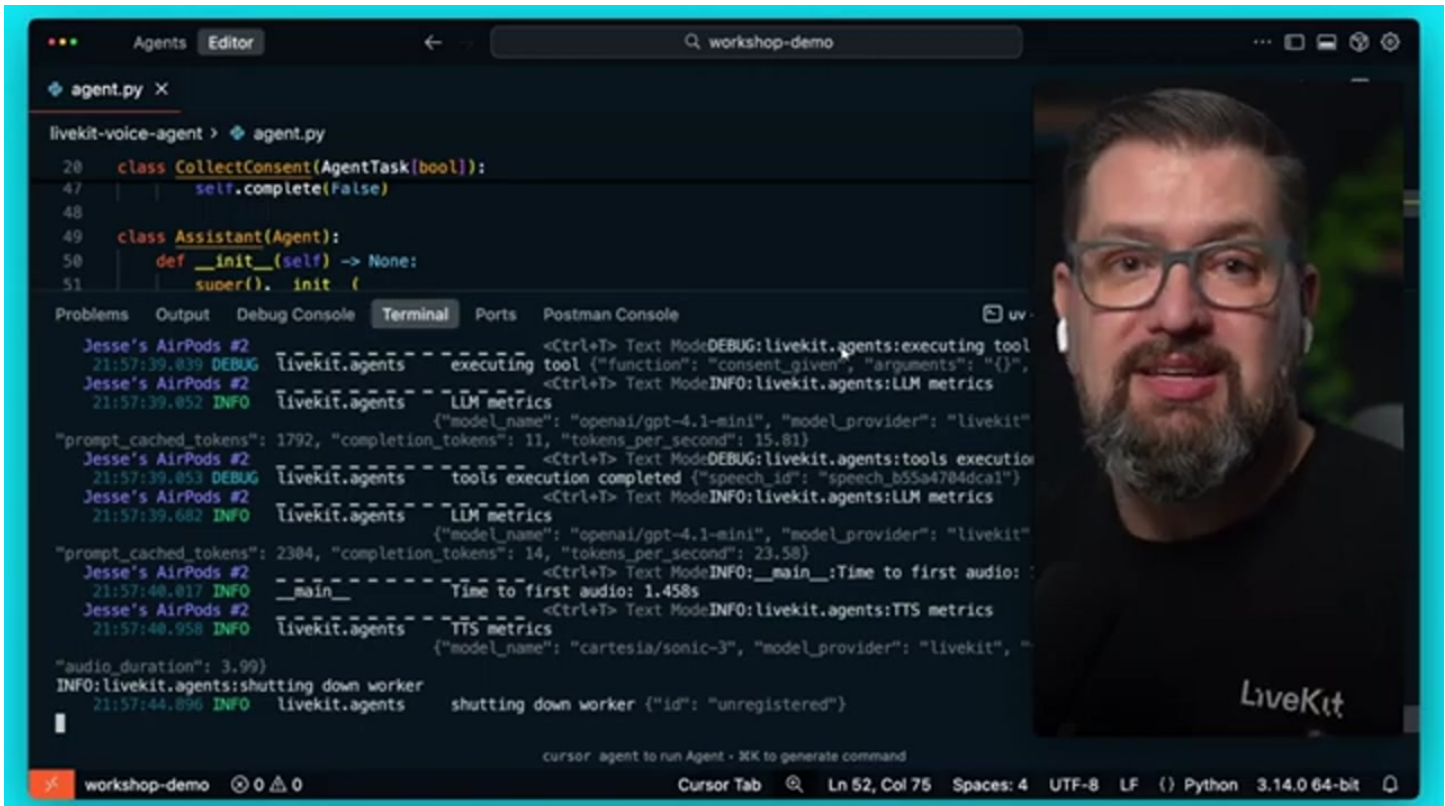
The screenshot shows a video call interface. On the right, a man with a beard and glasses is visible. On the left, a code editor window titled 'workshop-demo' is open. The editor shows a file named 'agent.py' with the following Python code:

```
livekit-agent > agent.py
20 class CollectConsent(AgentTask[bool]):
47     self.complete(False)
48
49 class Assistant(Agent):
50     def __init__(self) -> None:
51         super().__init__()
```

Below the code editor, a terminal window displays logs for 'livekit.agents'. The logs include timestamps, log levels (DEBUG, INFO), and messages such as 'using preemptive generation', 'EOU metrics', 'LLM metrics', and 'tools execution completed'. The logs also show model names like 'multilingual' and 'openai/gpt-4.1-mini'.

>> Yes. >> Thank you for your consent. How can I assist you today? >> Very good. So, it works. Now, if you're using LiveKit's agent observability features and you want session recordings to respect the user's consent choice, then you could start a new agent session with the record parameter set to false when uh consent is denied. This gives

■ [59:56 - 01:00:16]

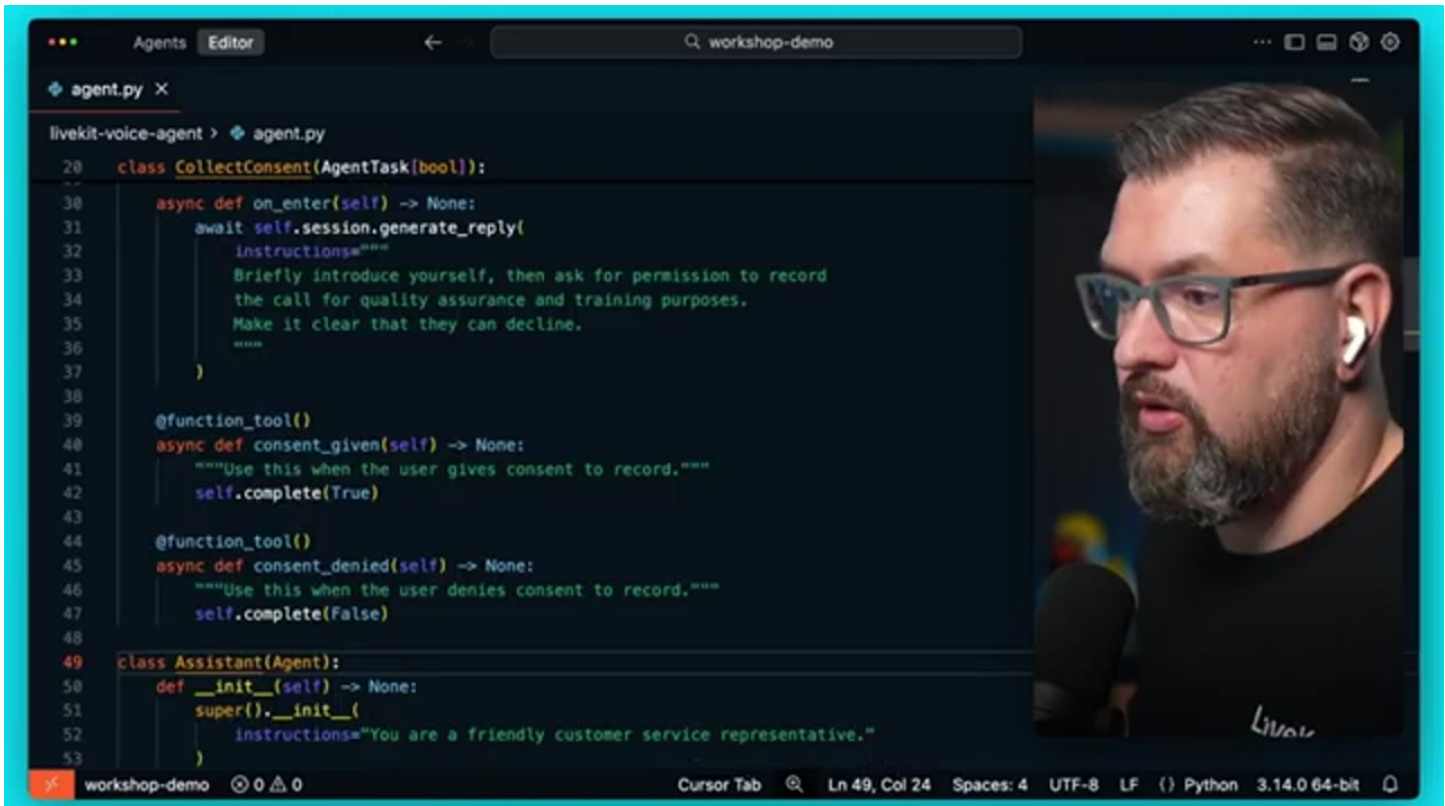


```
agent.py X
livekit-voice-agent > agent.py
20 class CollectConsent(AgentTask[bool]):
47     self.complete(False)
48
49 class Assistant(Agent):
50     def __init__(self) -> None:
51         super().init (

Problems Output Debug Console Terminal Ports Postman Console
Jesse's AirPods #2 21:57:39.839 DEBUG livekit.agents executing tool <Ctrl+T> Text ModeDEBUG:livekit.agents:executing tool
Jesse's AirPods #2 21:57:39.852 INFO livekit.agents LLM metrics {"function": "consent_given", "arguments": "{}",
Jesse's AirPods #2 21:57:39.852 INFO livekit.agents LLM metrics {"model_name": "openai/gpt-4.1-mini", "model_provider": "livekit",
"prompt_cached_tokens": 1792, "completion_tokens": 11, "tokens_per_second": 15.81)
Jesse's AirPods #2 21:57:39.853 DEBUG livekit.agents tools execution <Ctrl+T> Text ModeDEBUG:livekit.agents:tools execution
Jesse's AirPods #2 21:57:39.882 INFO livekit.agents LLM metrics {"speech_id": "speech_b55a4784dca1"}
Jesse's AirPods #2 21:57:39.882 INFO livekit.agents LLM metrics {"model_name": "openai/gpt-4.1-mini", "model_provider": "livekit",
"prompt_cached_tokens": 2384, "completion_tokens": 14, "tokens_per_second": 23.58)
Jesse's AirPods #2 21:57:40.017 INFO __main__ Time to first audio: 1.458s <Ctrl+T> Text ModeINFO:livekit.agents:TTS metrics
Jesse's AirPods #2 21:57:40.958 INFO livekit.agents TTS metrics {"model_name": "cartesia/sonic-3", "model_provider": "livekit",
"audio_duration": 3.99)
INFO:livekit.agents:shutting down worker
21:57:44.896 INFO livekit.agents shutting down worker {"id": "unregistered"}
```

when uh consent is denied. This gives you fine grain control over which sessions get recorded. So, check the agent observability docs for uh more details on session level recording settings. Now, what happens when a user says they want to talk to a manager or escalate in some way? Let's take a look at that. Let's go ahead and add another

■ [01:00:16 - 01:00:36]



The screenshot shows a code editor window titled 'workshop-demo' with a file named 'agent.py' open. The code defines two classes: 'CollectConsent' and 'Assistant'. The 'CollectConsent' class has an 'on_enter' method that generates a reply with instructions to introduce oneself and ask for consent. It also has two function tools, 'consent_given' and 'consent_denied', which complete the task with True or False respectively. The 'Assistant' class has an 'init' method that sets the instructions to 'You are a friendly customer service representative.'.

```
livekit-voice-agent > agent.py
20 class CollectConsent(AgentTask[bool]):
21     """
22     async def on_enter(self) -> None:
23         await self.session.generate_reply(
24             instructions="""
25             Briefly introduce yourself, then ask for permission to record
26             the call for quality assurance and training purposes.
27             Make it clear that they can decline.
28             """
29         )
30
31     @function_tool()
32     async def consent_given(self) -> None:
33         """Use this when the user gives consent to record."""
34         self.complete(True)
35
36     @function_tool()
37     async def consent_denied(self) -> None:
38         """Use this when the user denies consent to record."""
39         self.complete(False)
40
41 class Assistant(Agent):
42     def __init__(self) -> None:
43         super().__init__(
44             instructions="You are a friendly customer service representative."
45         )
```

On the right side of the editor, there is a video feed of a man with a beard and glasses, wearing a black t-shirt with the 'LiveKit' logo, speaking into a microphone.

at that. Let's go ahead and add another class. So, this is going to be our manager agent. And here's the instructions. You are a customer service uh manager. You handle escalated escalated issues that frontline agents could not resolve. Be empathetic and solution focused. You have authority to offer refunds, credits, or other accommodations. Very nice. And we can

■ [01:00:36 - 01:00:56]

```
class CollectConsent(AgentTask[bool]):
    @function_tool()
    async def consent_denied(self) -> None:
        """Use this when the user denies consent to record."""
        self.complete(False)

class ManagerAgent(Agent):
    def __init__(self, chat_ctx=None):
        super().__init__(
            instructions="""
            You are a customer service manager. You handle escalated issues
            that frontline agents couldn't resolve. Be empathetic and
            solution-focused. You have authority to offer refunds, credits,
            or other accommodations.
            """,
            chat_ctx=chat_ctx,
            tts="cartesia/sonic-3:6f84f4b8-58a2-430c-8c79-688dad597532",
        )

    async def on_enter(self) -> None:
        await self.session.generate_reply(
            instructions="""
            Introduce yourself as a manager. Acknowledge that the customer
            asked to speak with someone senior. Ask how you can help resolve
            their concern.
            """
```



accommodations. Very nice. And we can also change uh the the voice here. Of course, we want to make sure that the voice is different because the manager obviously is a different person. So, let's go back into our main assistant agent and let's update the instructions. All right. So, you're a friendly customer service representative. Help customers with general inquiries. If

■ [01:00:56 - 01:01:16]

```
py ●
ice-agent > agent.py

class ManagerAgent(Agent):
    async def on_enter(self) -> None:

class Assistant(Agent):
    def __init__(self) -> None:
        super().__init__(
            instructions="""
            You are a friendly customer service representative. Help customers
            with general inquiries. If they ask for a manager or you can't
            resolve their issue, use the escalate_to_manager tool.
            """
        )

    async def on_enter(self) -> None:
        consent = await CollectConsent(chat_ctx=self.chat_ctx)

        if consent:
            await self.session.generate_reply(
                instructions="Thank them and offer your assistance."
            )
        else:
            await self.session.generate_reply(
```



customers with general inquiries. If they ask for a manager or you can't resolve their issue, use the escalate to manager tool. And so, let's go ahead and add that tool now. So escalate to manager uh is going to be transfer the customer to a manager when requested or you can't resolve their in their issue

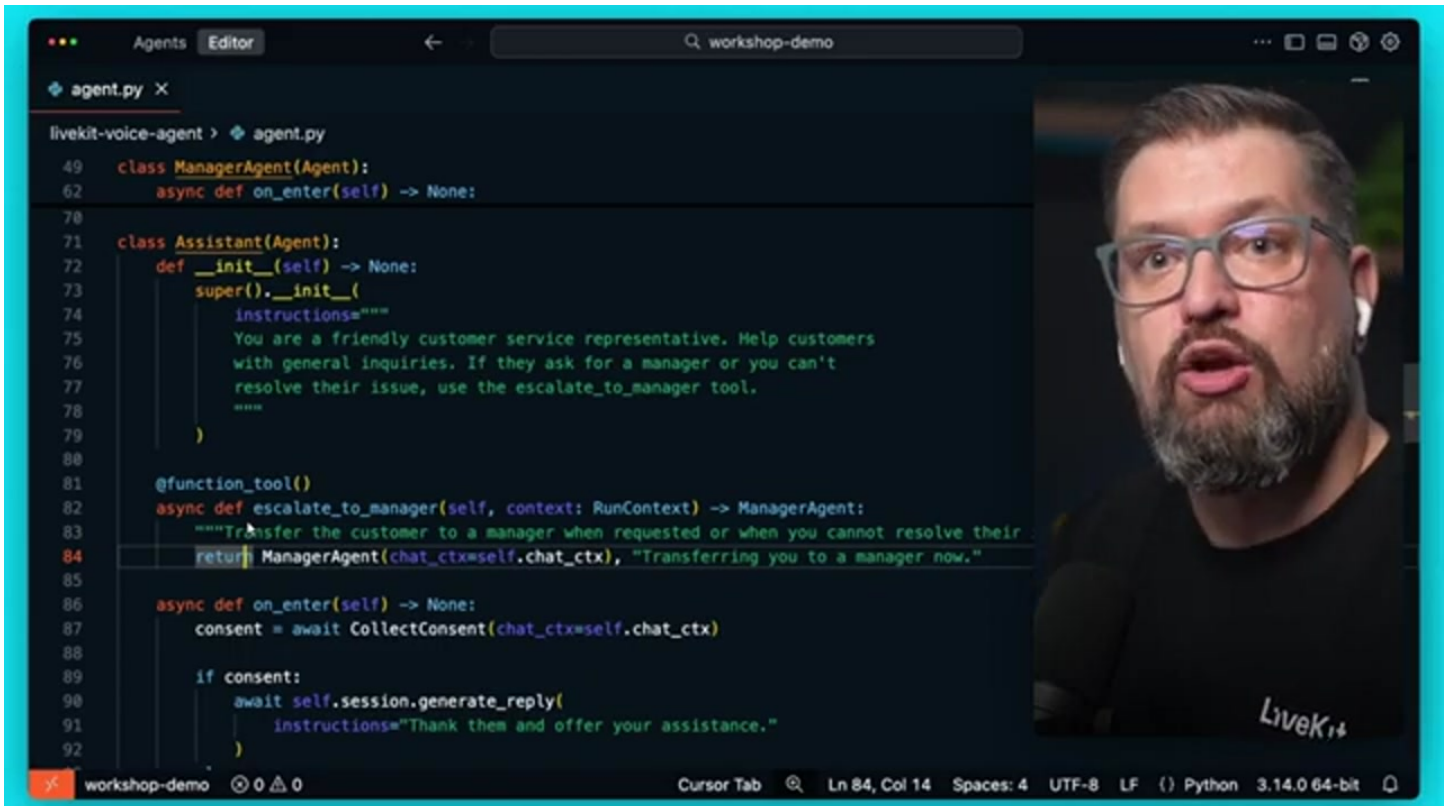
■ [01:01:16 - 01:01:36]

```
49 class ManagerAgent(Agent):
62     async def on_enter(self) -> None:
70
71 class Assistant(Agent):
72     def __init__(self) -> None:
73         super().__init__(
74             instructions="""
75             You are a friendly customer service representative. Help customers
76             with general inquiries. If they ask for a manager or you can't
77             resolve their issue, use the escalate_to_manager tool.
78             """
79         )
80
81     @function_tool()
82     async def escalate_to_manager(self, context: RunContext) -> ManagerAgent:
83         """Transfer the customer to a manager when requested or when you cannot reso
84         return ManagerAgent(chat_ctx=self.chat_ctx), "Transferring you to a manager r
85
86     async def on_enter(self) -> None:
87         consent = await CollectConsent(chat_ctx=self.chat_ctx)
88
89         if consent:
90             await self.session.generate_reply(
91                 instructions="Thank them and offer your assistance."
92             )
```



and return manager agent. So it's going to call that manager agent and say transferring you to a manager now. And so when a tool uh returns a new agent as the first value in the tuple, the session automatically hands off control to that agent. And the second value there is the message that is going to speak before that handoff completes.

■ [01:01:36 - 01:01:56]

The image is a screenshot of a video call. On the right side, there is a video feed of a man with a beard and glasses, wearing a black t-shirt with the 'LiveKit' logo. He is speaking into a microphone. On the left side, there is a code editor window titled 'Agents Editor' with a search bar containing 'workshop-demo'. The editor shows a file named 'agent.py' with the following Python code:

```
livekit-voice-agent > agent.py
49 class ManagerAgent(Agent):
62     async def on_enter(self) -> None:
70
71 class Assistant(Agent):
72     def __init__(self) -> None:
73         super().__init__(
74             instructions="""
75             You are a friendly customer service representative. Help customers
76             with general inquiries. If they ask for a manager or you can't
77             resolve their issue, use the escalate_to_manager tool.
78             """
79         )
80
81 @function_tool()
82 async def escalate_to_manager(self, context: RunContext) -> ManagerAgent:
83     """Transfer the customer to a manager when requested or when you cannot resolve their
84     return ManagerAgent(chat_ctx=self.chat_ctx), "Transferring you to a manager now."
85
86 async def on_enter(self) -> None:
87     consent = await CollectConsent(chat_ctx=self.chat_ctx)
88
89     if consent:
90         await self.session.generate_reply(
91             instructions="Thank them and offer your assistance."
92         )
```

The status bar at the bottom of the editor shows 'workshop-demo', 'Cursor Tab', 'Ln 84, Col 14', 'Spaces: 4', 'UTF-8', 'LF', 'Python', and '3.14.0 64-bit'.

speak before that handoff completes. Now, by default, each agent starts fresh, but you usually want the manager to know what's already been discussed. So, we want to make sure that that conversation history persists. So, notice uh in the escalation tool here, uh in the manager agent, we're actually passing in that context here to the new

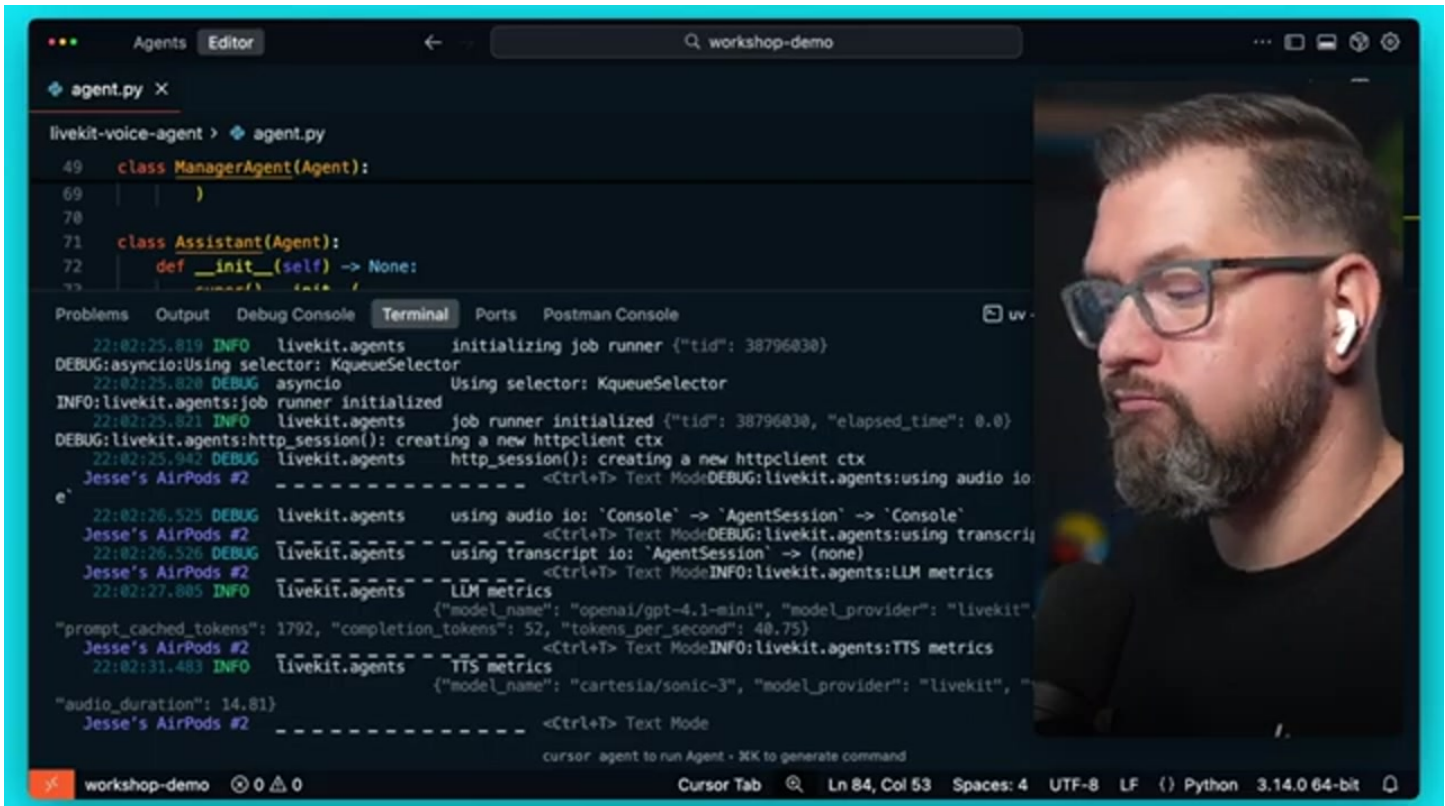
■ [01:01:56 - 01:02:16]

```
49 class ManagerAgent(Agent):
62     async def on_enter(self) -> None:
70
71 class Assistant(Agent):
72     def __init__(self) -> None:
73         super().__init__(
74             instructions="""
75             You are a friendly customer service representative. Help customers
76             with general inquiries. If they ask for a manager or you can't
77             resolve their issue, use the escalate_to_manager tool.
78             """
79         )
80
81     @function_tool()
82     async def escalate_to_manager(self, context: RunContext) -> ManagerAgent:
83         """Transfer the customer to a manager when requested or when you cannot resolve
84         return ManagerAgent(chat_ctx=self.chat_ctx), "Transferring you to a manager"
85
86     async def on_enter(self) -> None:
87         consent = await CollectConsent(chat_ctx=self.chat_ctx)
88
89         if consent:
90             await self.session.generate_reply(
91                 instructions="Thank them and offer your assistance."
92             )
```



passing in that context here to the new agent. This carries the full conversation history to the manager. so they can pick up right where the previous agent left off. Let's give this one a whirl. >> Hello. Before we proceed, may I kindly ask for your permission to record this call for quality assurance and training

■ [01:02:16 - 01:02:36]



The screenshot shows the Cursor IDE interface. The top bar indicates the file is 'agent.py' in the 'livekit-voice-agent' project. The editor shows the following Python code:

```
49 class ManagerAgent(Agent):
69     )
70
71 class Assistant(Agent):
72     def __init__(self) -> None:
73         pass
```

The terminal displays the following logs:

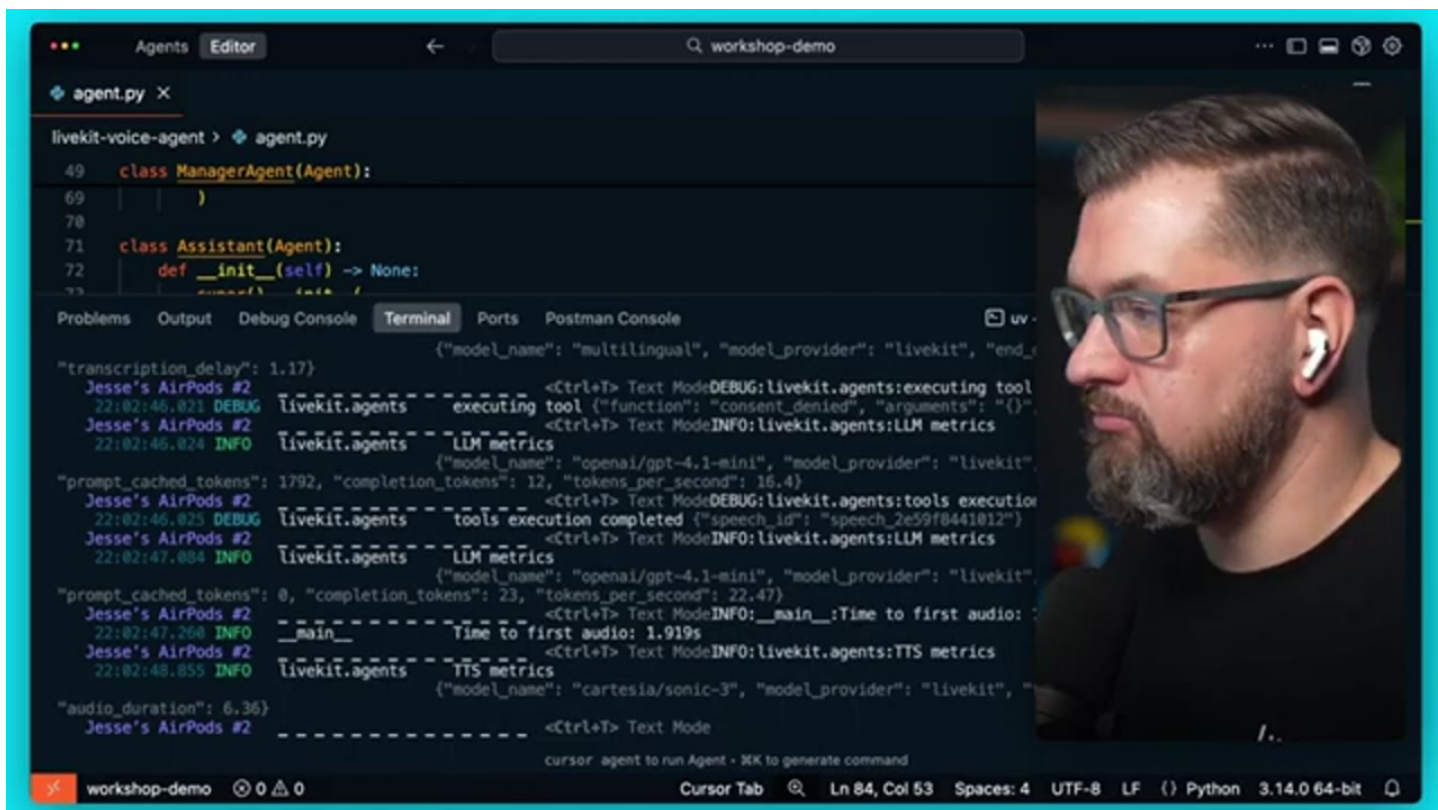
```
22:02:25.819 INFO livekit.agents initializing job runner {"tid": 38796030}
DEBUG:asyncio:Using selector: KqueueSelector
22:02:25.820 DEBUG asyncio Using selector: KqueueSelector
INFO:livekit.agents:job runner initialized
22:02:25.821 INFO livekit.agents job runner initialized {"tid": 38796030, "elapsed_time": 0.0}
DEBUG:livekit.agents:http_session(): creating a new httpclient ctx
22:02:25.942 DEBUG livekit.agents http_session(): creating a new httpclient ctx
Jesse's AirPods #2 ----- <Ctrl+T> Text ModeDEBUG:livekit.agents:using audio io
e'
22:02:26.525 DEBUG livekit.agents using audio io: 'Console' -> 'AgentSession' -> 'Console'
Jesse's AirPods #2 ----- <Ctrl+T> Text ModeDEBUG:livekit.agents:using transcrip
22:02:26.526 DEBUG livekit.agents using transcript io: 'AgentSession' -> (none)
Jesse's AirPods #2 ----- <Ctrl+T> Text ModeINFO:livekit.agents:LLM metrics
22:02:27.005 INFO livekit.agents LLM metrics
{"model_name": "openai/gpt-4.1-mini", "model_provider": "livekit",
"prompt_cached_tokens": 1792, "completion_tokens": 52, "tokens_per_second": 40.75}
Jesse's AirPods #2 ----- <Ctrl+T> Text ModeINFO:livekit.agents:TTS metrics
22:02:31.483 INFO livekit.agents TTS metrics
{"model_name": "cartesia/sonic-3", "model_provider": "livekit",
"audio_duration": 14.81}
Jesse's AirPods #2 ----- <Ctrl+T> Text Mode
```

The status bar at the bottom shows 'workshop-demo', '0 0 0', 'Cursor Tab', 'Ln 84, Col 53', 'Spaces: 4', 'UTF-8', 'LF', 'Python', and '3.14.0 64-bit'.



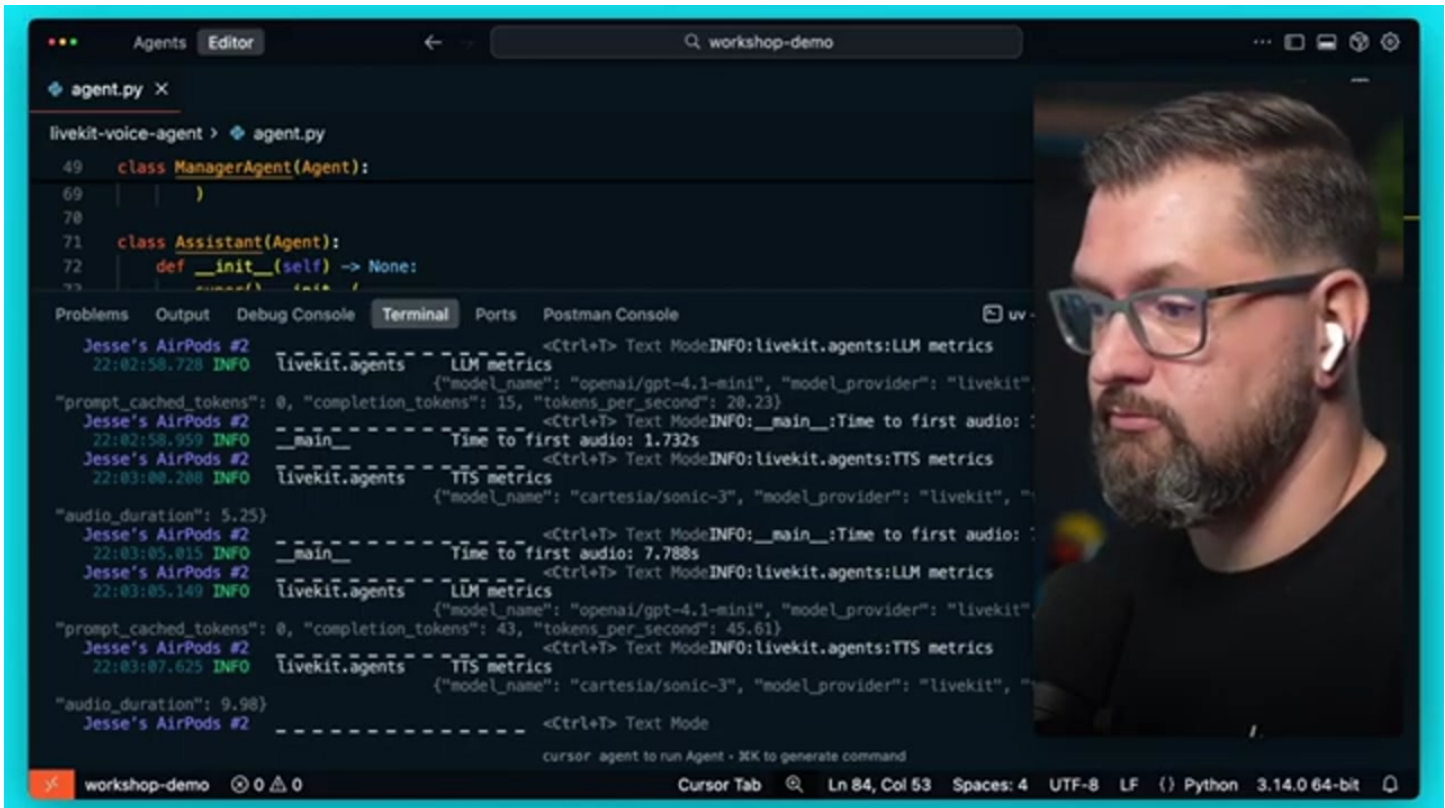
call for quality assurance and training purposes. Your consent is entirely voluntary and you may decline if you prefer. Could you please let me know if you agree to this recording? >> No. Thank you for letting me know. I understand and we'll proceed without recording. How can I assist you today? I

■ [01:02:36 - 01:02:56]



recording. How can I assist you today? I really need to speak with a manager right now, please. >> I am transferring you to a manager now. They will assist you shortly. >> Hello, I'm the customer service manager. I understand you asked to speak with someone senior. I'm here to help. Could you please share the details of your concern so I can assist in resolving it?

■ [01:02:56 - 01:03:16]



concern so I can assist in resolving it? No concerns. I just wanted to speak to you. Thanks. Bye. How easy was that to get to a manager? Now, for complex workflows with multiple steps like collecting shipping information, uh you can use a task group. A task group lets you define multiple tasks that run in sequence. Each task has an ID and a

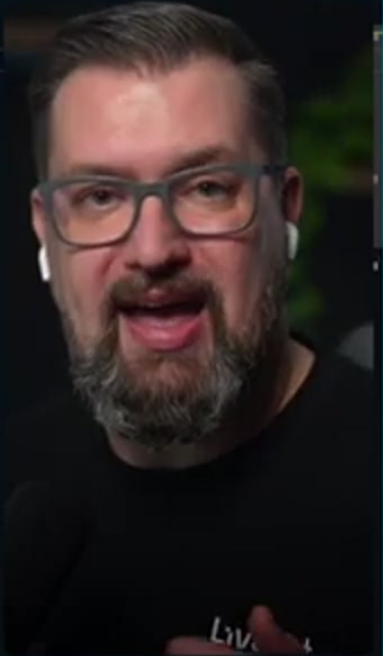
Multi-step workflows



LIVEKIT.IO

sequence. Each task has an ID and a description. And when the group completes, you can then access the results by that ID. And here is an example of a task group that collects both an email and shipping address. So first we're going to import our task group from agents beta workflows and then our agent task function tool and run context like we have in the past as

■ [01:03:36 - 01:03:56]



```
agent.py x
livekit-voice-agent > agent.py
1 from livekit.agents.beta.workflows import TaskGroup
2 from livekit.agents import AgentTask, function_tool, RunContext
3 from dataclasses import dataclass
4
5 @dataclass
6 class EmailResult:
7     email_address: str
8
9 @dataclass
10 class AddressResult:
11     address: str
12
13 class GetEmailTask(AgentTask[EmailResult]):
14     def __init__(self) -> None:
15         super().__init__(
16             instructions="Collect the user's email address."
17         )
18
19     @function_tool()
20     async def record_email(self, context: RunContext, email: str) -> None:
21         """Record the user's email address"""
22         self.complete(EmailResult(email_address=email))
23
24 class GetAddressTask(AgentTask[AddressResult]):
25     def __init__(self) -> None:
26         super().__init__(
27             instructions="Collect the user's shipping address."
28         )
```

workshop-demo 0 0 0 Cursor Tab Ln 2, Col 60 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit

run context like we have in the past as well as this data class to define our uh email result and address result. And then we're just going to define two simple tasks. We have a get email task and a get address task. The get email task collects an email address and stores it in a data class. And then the um get address task does the same thing

■ [01:03:56 - 01:04:16]

```
class AddressResult:
    address: str

class GetEmailTask(AgentTask[EmailResult]):
    def __init__(self) -> None:
        super().__init__(
            instructions="Collect the user's email address."
        )

    @function_tool()
    async def record_email(self, context: RunContext, email: str) -> None:
        """Record the user's email address"""
        self.complete(EmailResult(email_address=email))

class GetAddressTask(AgentTask[AddressResult]):
    def __init__(self) -> None:
        super().__init__(
            instructions="Collect the user's shipping address."
        )

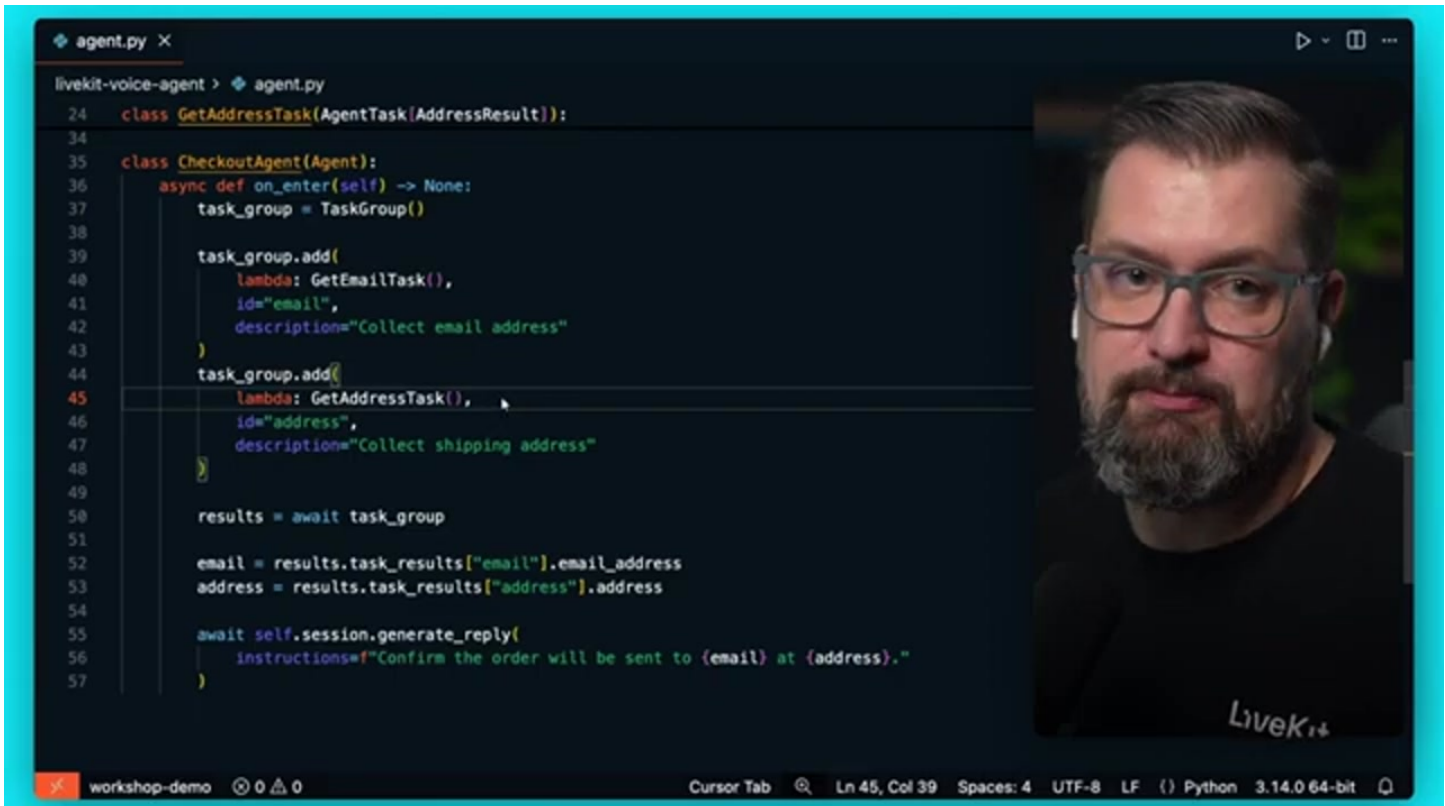
    @function_tool()
    async def record_address(self, context: RunContext, address: str) -> None:
        """Record the user's shipping address"""
        self.complete(AddressResult(address=address))

class CheckoutAgent(Agent):
```



um get address task does the same thing for the shipping address. And then in the checkout agent, uh we're going to create a task group. And then we're going to add both of those to the task group. Along with that, we will add an ID and a description for each. And each task is wrapped in a lambda so it can be uh reinitialized if the user needs to go back and make a correction. And then we

■ [01:04:16 - 01:04:36]



```
agent.py X
livekit-voice-agent > agent.py
24 class GetAddressTask(AgentTask[AddressResult]):
34
35 class CheckoutAgent(Agent):
36     async def on_enter(self) -> None:
37         task_group = TaskGroup()
38
39         task_group.add(
40             lambda: GetEmailTask(),
41             id="email",
42             description="Collect email address"
43         )
44         task_group.add(
45             lambda: GetAddressTask(),
46             id="address",
47             description="Collect shipping address"
48         )
49
50         results = await task_group
51
52         email = results.task_results["email"].email_address
53         address = results.task_results["address"].address
54
55         await self.session.generate_reply(
56             instructions=f"Confirm the order will be sent to {email} at {address}."
57         )
```

workshop-demo 0 0 0 Cursor Tab Ln 45, Col 39 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit

back and make a correction. And then we just await that task group. And then it runs each task in sequence. And once complete, we access the results by their task ID and confirm the order details with the user. So one of the great things about task groups is they support going back to previous steps if the user needs to make corrections. So this makes multi-step data collection feel natural.

■ [01:04:36 - 01:04:56]



```
agent.py x
livekit-voice-agent > agent.py
24 class GetAddressTask(AgentTask(AddressResult)):
34
35 class CheckoutAgent(Agent):
36     async def on_enter(self) -> None:
37         task_group = TaskGroup()
38
39         task_group.add(
40             lambda: GetEmailTask(),
41             id="email",
42             description="Collect email address"
43         )
44         task_group.add(
45             lambda: GetAddressTask(),
46             id="address",
47             description="Collect shipping address"
48         )
49
50         results = await task_group
51
52         email = results.task_results["email"].email_address
53         address = results.task_results["address"].address
54
55         await self.session.generate_reply(
56             instructions=f"Confirm the order will be sent to {email} at {address}."
57         )
```

workshop-demo 0 0 0 Cursor Tab Ln 56, Col 84 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit

multi-step data collection feel natural. And then for the ultimate escalation, uh you might need to transfer to an actual human agent. And so this typically involves SIP or telefan. So, first you'll want to disable interruptions so that the transfer message plays fully and then use the SIP integration to dial out to your phone system and bridge to a

■ [01:04:56 - 01:05:16]

Warm handoff to a human



LIVEKIT.IO

out to your phone system and bridge to a human agent. Now, SIP and telefan configuration is its own topic uh with more setup involved. And so, we'll have a separate videos that go deeper into telefan that's not going to be covered in this workshop. And so, don't worry if this part seems a little complex. For now, I'll show you the patterns for how human handoff works. And that would look

Warm handoff to a human



LiveKit

LIVEKIT.IO

human handoff works. And that would look like this. So we're going to import uh function tool run context uh and get a job context and then define this tool transfer to human and then transfer the call to a human agent. So we're going to disallow the interruption and then we're going to say I'm transferring you to a human agent now. Please hold on. So then

■ [01:05:36 - 01:05:56]

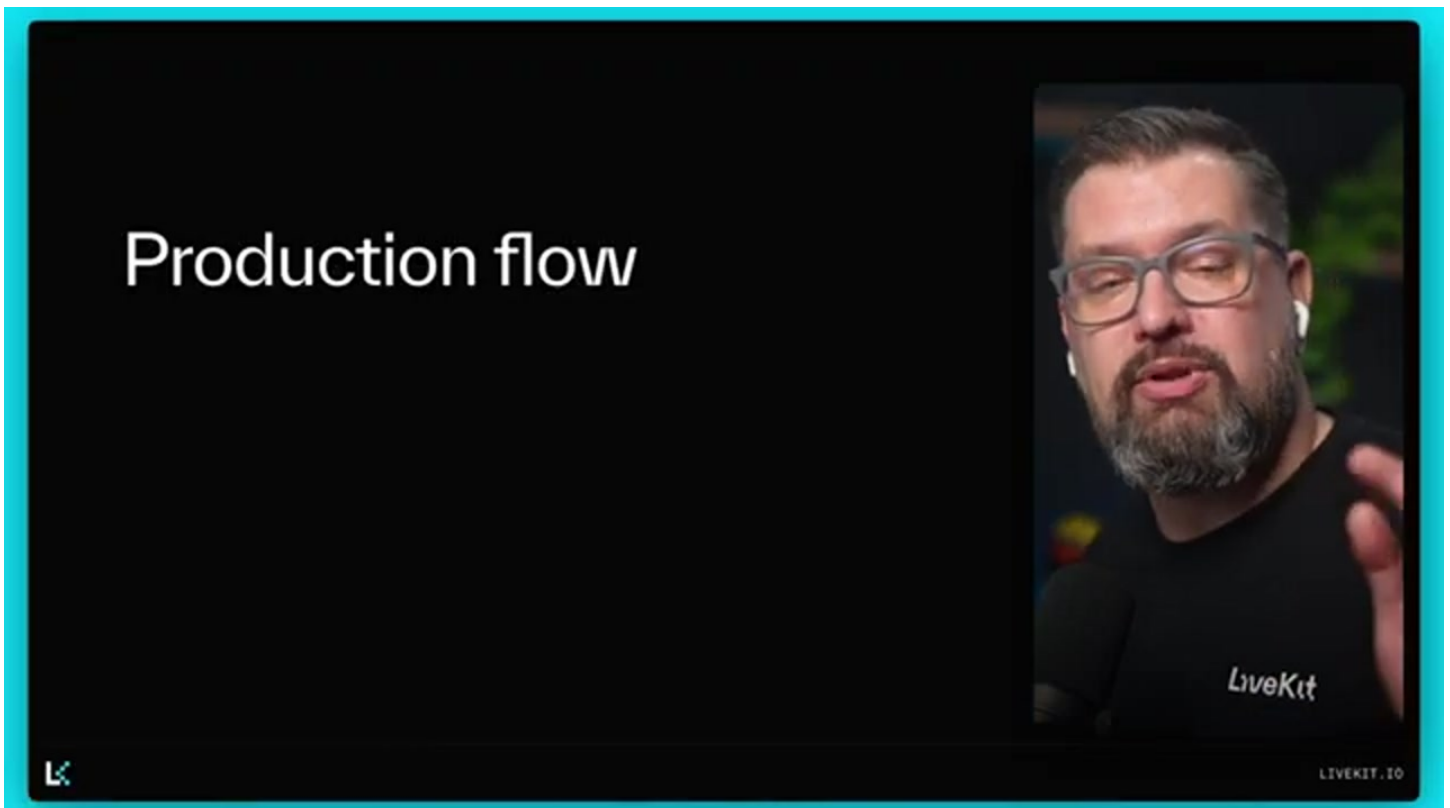
```
agent.py
livekit-voice-agent > agent.py

15
16 from livekit.agents import function_tool, RunContext, get_job_context
17
18 @function_tool()
19 async def transfer_to_human(self, context: RunContext) -> None:
20     """Transfer the call to a human agent."""
21     context.disallow_interruptions()
22
23     await context.session.say(
24         "I'm transferring you to a human agent now. Please hold."
25     )
26
27     room = get_job_context().room
28     await room.local_participant.publish_sip_participant(
29         sip_trunk_id="your-trunk-id",
30         dial_to="sip:support@your-pbx.com",
31     )
32
33
34
35
36
37
```



human agent now. Please hold on. So then we get the room context and then we await this uh publish SIP participant including the the SIP trunk ID and the dial to uh information that is specific to your SIP configuration. So then the SIP integration dials out to your phone system and bridges in the human agent. So a complete production uh agent flow

■ [01:05:56 - 01:06:16]



So a complete production uh agent flow might look like this. First, the consent task collects uh recording permission and then a triage agent determines the user's needs and then a specialized agent handles the specific request whether it's billing, support or sales

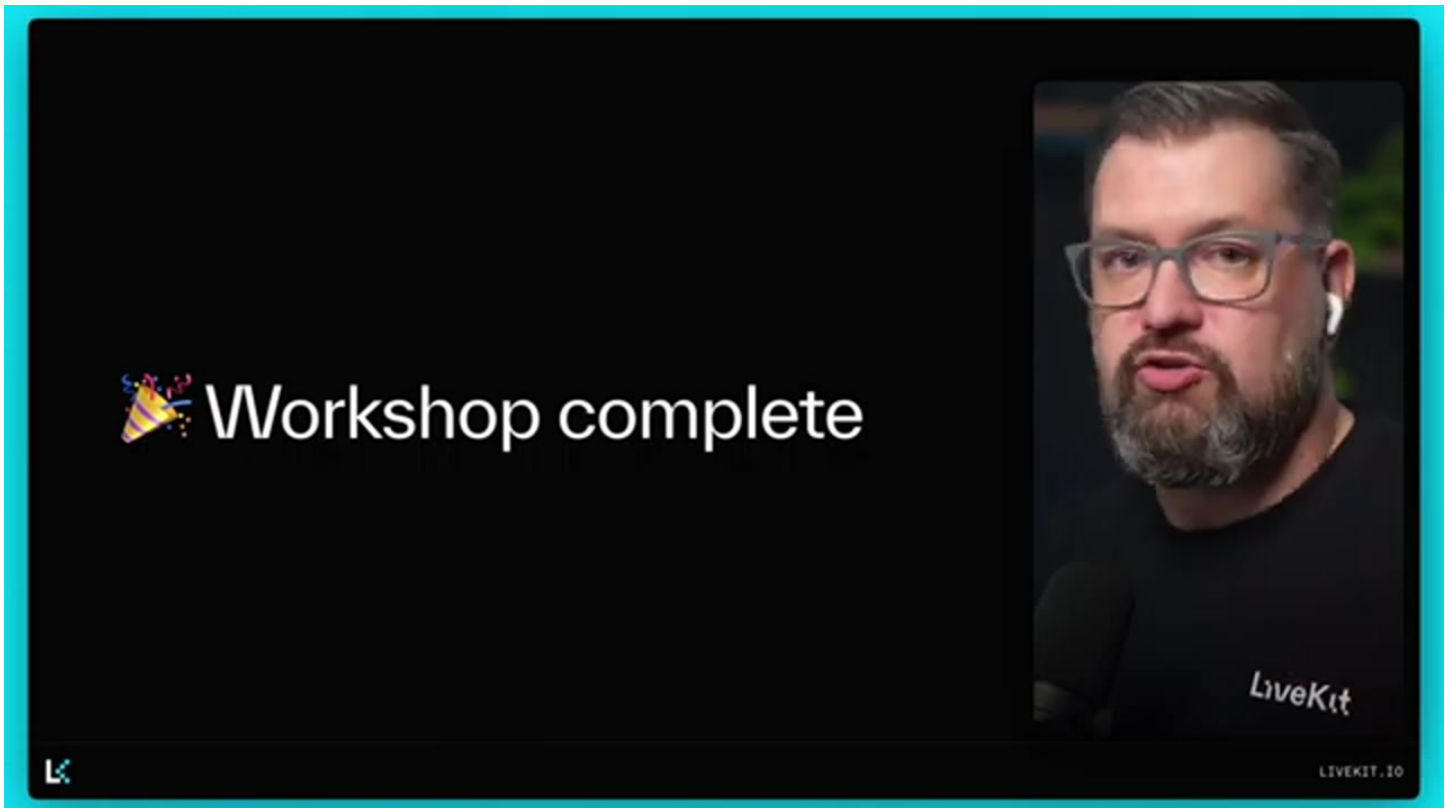
Production flow

- Consent task
- Triage agent
- Specialized agent



whether it's billing, support or sales and then escalations to a manager occurs uh as needed and then a human handoff is the last resort. Each transition preserves relevant context while allowing agents to have uh different personalities, tools, and permissions. Congratulations. You built a production quality voice agent from scratch. You

■ [01:06:36 - 01:06:56]



quality voice agent from scratch. You covered ST, LLM, TTS, and why Web RTC matters for real-time audio. You you learned semantic turn detection to prevent awkward interruptions. You crafted system prompts, selected voices, and built fallback adapters for resilience. You measured performance and

What you built

- Voice agent foundations
- Semantic turn detection
- Personality & fallbacks
- Metrics & preemptive generation



LIVEKIT.IO

resilience. You measured performance and enabled preemptive generation. You connected your agent to external APIs and MCP servers. And you built production workflows with tasks and agent handoffs. Your agent can now listen in real time, respond naturally, call external services, collect structured information, and hand off to specialized agents, or even humans when

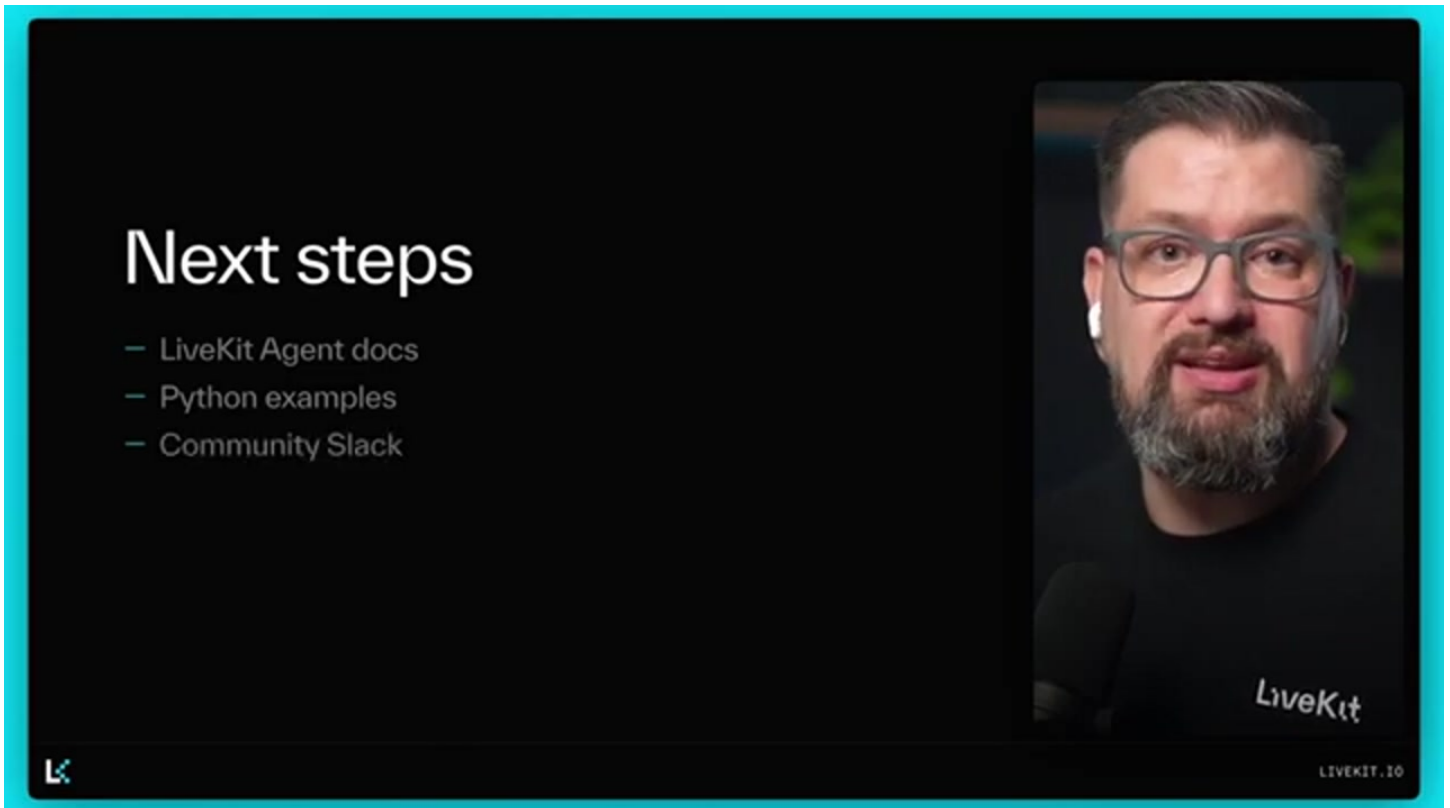
What you built

- Voice agent foundations
- Semantic turn detection
- Personality & fallbacks
- Metrics & preemptive generation
- Tools & MCP integration
- Consent & escalations



LIVEKIT.IO

specialized agents, or even humans when needed. So, here is where you go next. Browse the Lifegate agents documentation for advanced topics like telefan, video processing, and real-time models. Explore the Python agents example repository for more complex workflows. Join the LifeKit community Slack to ask



Next steps

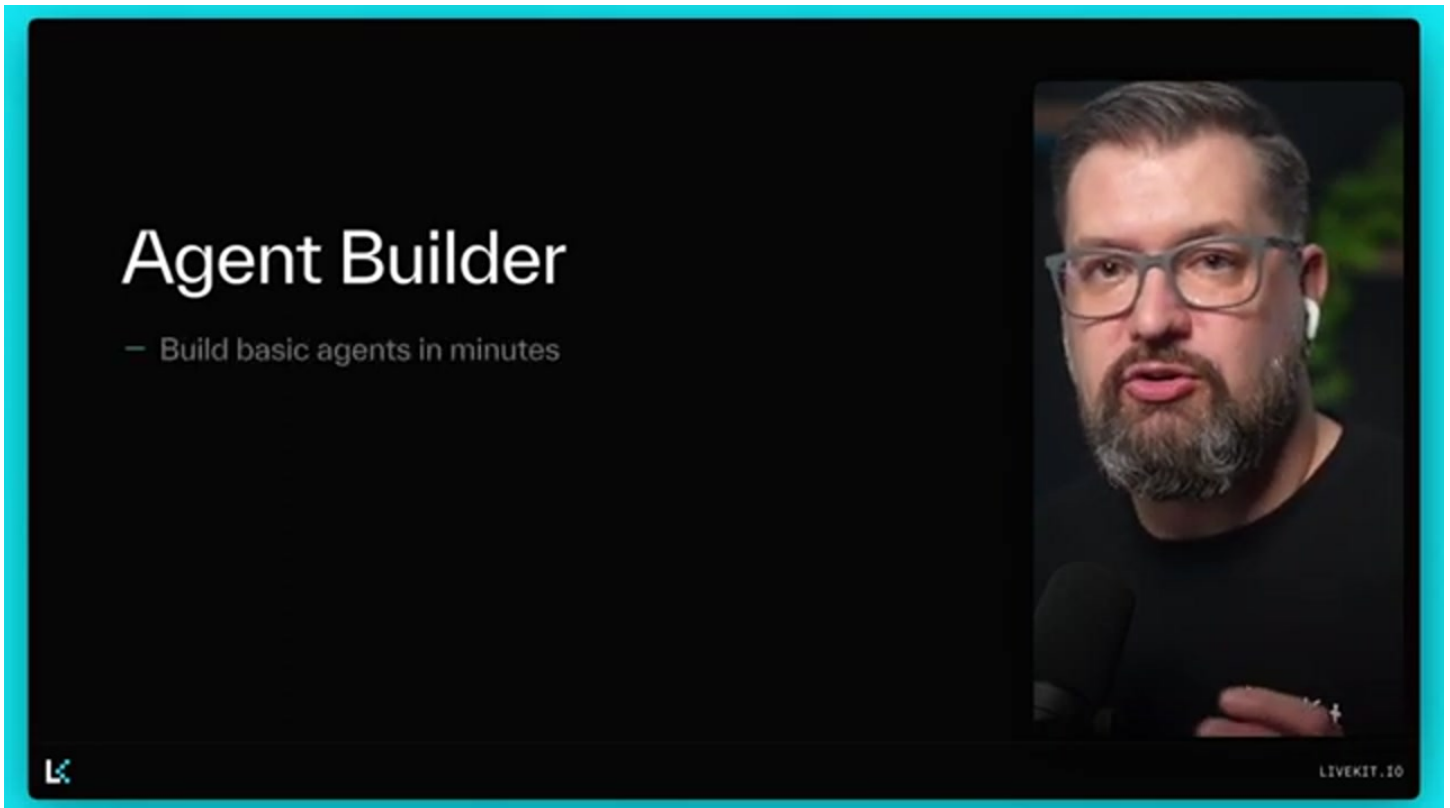
- LiveKit Agent docs
- Python examples
- Community Slack

LIVEKIT

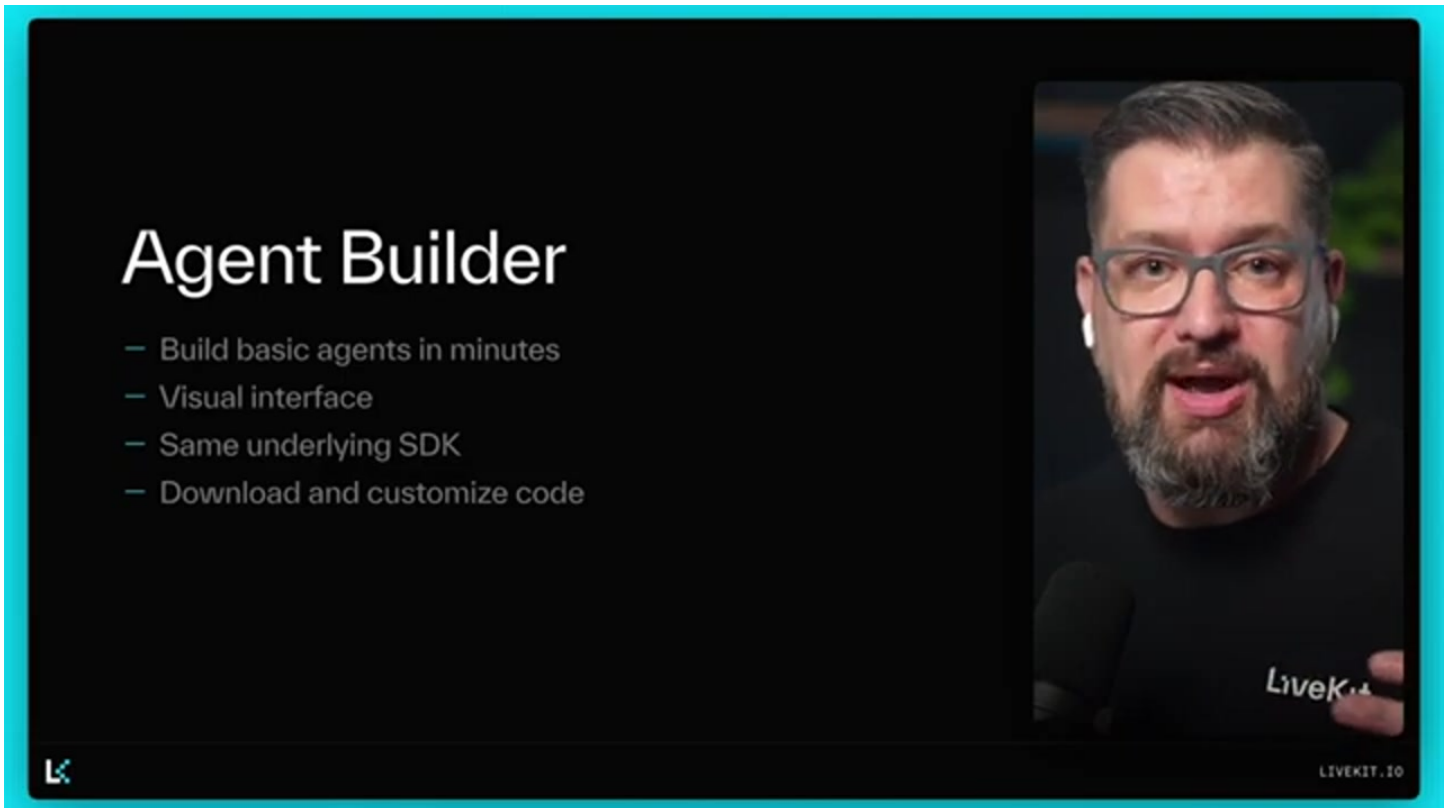
LIVEKIT.IO

Join the LifeKit community Slack to ask questions and share what you're building. And be sure to check out LifeKit Cloud for production hosting with built-in observability like we looked at. Links to all of these things are in the video description. And one more thing that I want to introduce you to is the LifeKit Cloud Agent Builder. Agent Builder is a visual tool that generates a basic voice agent in

■ [01:07:56 - 01:08:16]



generates a basic voice agent in minutes. You configure the providers through the web interface. You set the system prompts and it generates the Python code for you that you can then download and even deploy directly from the interface. The code that agent builder generates is yours. You can download it, open it up in your own editor, customize it, add the tools, the



The slide features a dark background with a cyan border. On the left, the title 'Agent Builder' is displayed in a large, white, sans-serif font. Below the title, a bulleted list of four features is shown in a smaller, white, sans-serif font. On the right side of the slide, there is a video overlay of a man with a beard and glasses, wearing a black t-shirt with the 'LiveKit' logo. He is speaking into a microphone. In the bottom left corner of the slide, there is a small 'LK' logo, and in the bottom right corner, the text 'LIVEKIT.IO' is visible.

Agent Builder

- Build basic agents in minutes
- Visual interface
- Same underlying SDK
- Download and customize code

LK LIVEKIT.IO

editor, customize it, add the tools, the workflows, and the custom logic that you learned in this workshop. Agent Builder gives you a great starting point. Everything that you learned here applies uh to extending that code. Now, why didn't we just start with Agent Builder? Well, it would have been much easier, yes, but uh I wanted you to see exactly how to start from scratch. Agent Builder

Agent Builder

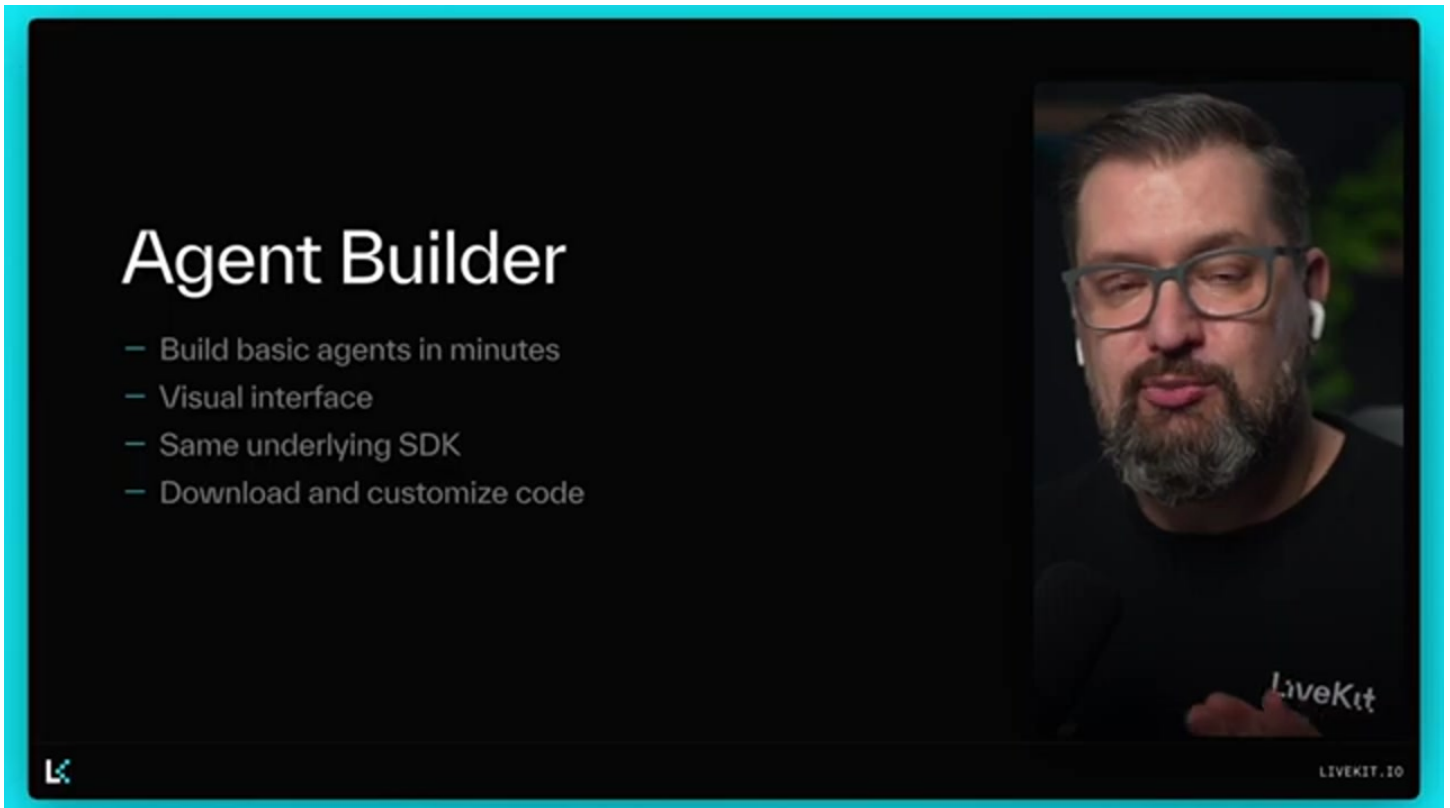
- Build basic agents in minutes
- Visual interface
- Same underlying SDK
- Download and customize code



LIVEKIT.IO

how to start from scratch. Agent Builder can get you up and running quickly. Now, beyond the basics, you can also configure tools directly in the builder interface and test your agent with a built-in test interface before deploying. And when you're ready, oneclick deploy pushes your agent to LiveKit Cloud. If you want a full walkthrough of agent builder

■ [01:08:56 - 01:09:16]



The video frame shows a presentation slide with a black background and white text. The title 'Agent Builder' is prominently displayed at the top left. Below it, a bulleted list outlines four key features: building basic agents in minutes, a visual interface, a shared underlying SDK, and the ability to download and customize code. In the top right corner, there is a small inset video of a man with a beard and glasses, wearing a black t-shirt with the 'LiveKit' logo. The bottom left of the slide features a small 'LK' logo, and the bottom right corner displays the website 'LIVEKIT.IO'.

Agent Builder

- Build basic agents in minutes
- Visual interface
- Same underlying SDK
- Download and customize code

LK LIVEKIT.IO

walkthrough of agent builder specifically, go check out our other video tutorial on that. I'll leave a link in the video description. You can find agent builder in your LifeKit cloud dashboard under agents. So go build a basic agent there, download the code, and you'll recognize those same patterns and components that you built from scratch here in this workshop. Now, I hope this workshop was helpful. Uh thank

■ [01:09:16 - 01:09:19]

Agent Builder

- Build basic agents in minutes
- Visual interface
- Same underlying SDK
- Download and customize code



LIVEKIT.IO

you for following along. Now go build something